Lab 4: Simple Python Database Connections

Name	Mehmoona Abid
Roll number	2023-BCS-097
Section	A
GitHub repository link	https://github.com/mehmoonaabid/DevOps

Estimated duration: 3 hours
Starter repository: Databases — fork this repository first
Submission repository: DevOps
Submission directory: Labs/Lab04
Screenshot directory: Labs/Lab04/screenshots

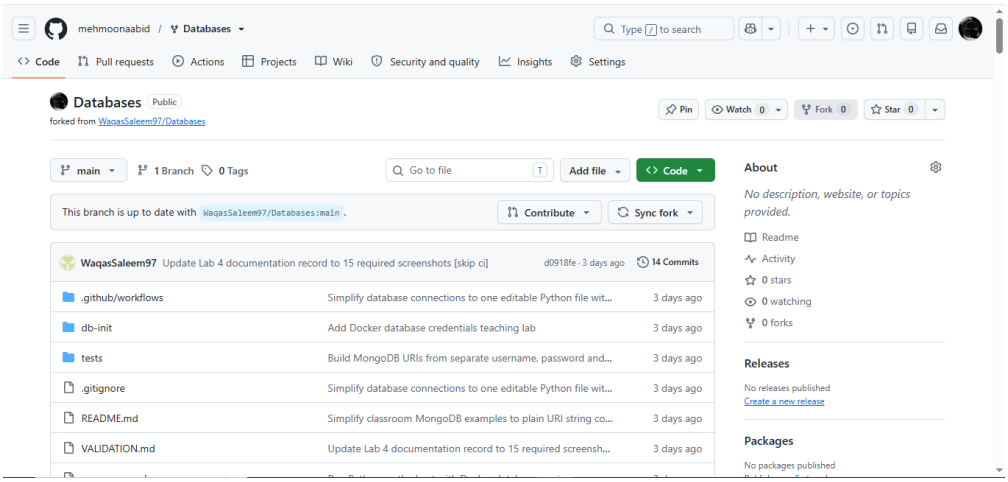
Getting Started

1. Create the evidence directory in your existing DevOps repository:
2. cd ~/DevOps

   mkdir -p Labs/Lab04/screenshots
3. Do the programming work in your Databases fork. Save screenshots in DevOps/Labs/Lab04/screenshots/ and include your observations and answers in Lab4_Solution.pdf.

Task 1: Fork and Clone the Databases Repository

1. Open Databases, select **Fork**, and keep the name Databases.
2. Confirm that the page shows <your-username>/Databases and the original repository as its source.
    - Take a split-screen screenshot as databases_fork.png.

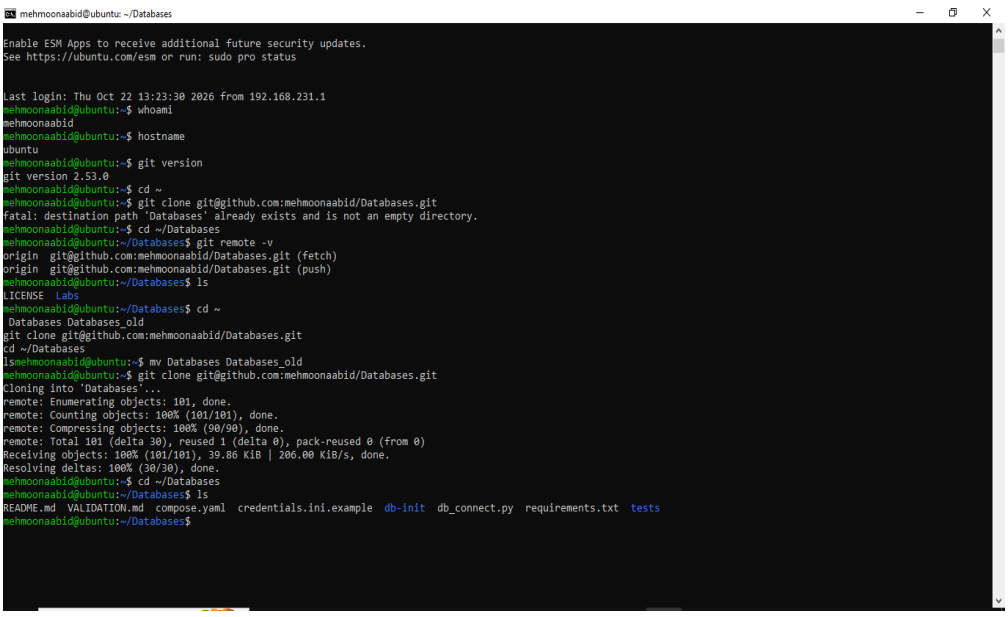


3. Clone your own fork:
4. cd ~
5. git clone git@github.com:<your-username>/Databases.git

cd Databases

Replace <your-username> with your GitHub username. Reuse an existing clone if you already have one.

- Screenshot as databases_clone.png.



Screenshot Required:

Task 2: Install Docker, Docker Compose, and Python on Ubuntu

1. Connect to your assigned Ubuntu server.
2. Update the Ubuntu package index and install Docker Engine and curl. On Ubuntu, the required package name is docker.io:
3. sudo apt update -y

sudo apt install -y docker.io curl

4. Create the system-wide Docker CLI plug-in directory and download the Docker Compose plug-in for an x86_64 Ubuntu server:
5. sudo mkdir -p /usr/local/lib/docker/cli-plugins
6. sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o /usr/local/lib/docker/cli-plugins/docker-compose

sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

7. Start the Docker service and add your current Ubuntu account to the docker group:
8. sudo systemctl start docker
9. sudo usermod -aG docker $USER

exit

Completely sign out of the Ubuntu server and reconnect so that the new group membership takes effect. Opening only a new terminal tab is not sufficient.

10. Reconnect and verify the Git, Docker, and Docker Compose installations:
11. ssh <github-username>@<ubuntu-server-ip>
12. git --version

13.docker --version

docker compose version

Do not continue until all three version commands display their installed versions.

- Save a screenshot showing the three commands and their installed versions as docker_installation.png.

Install Python and Prepare the Host Environment

sudo apt install -y python3 python3-venv python3-pip

cd ~/Databases

python3 -m venv .venv

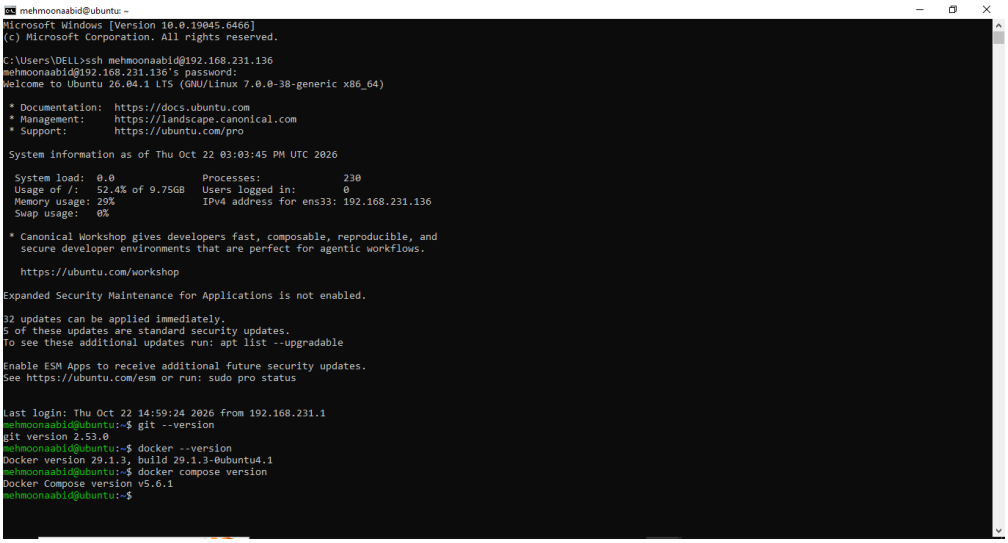
source .venv/bin/activate

python -m pip install -r requirements.txt

Edit db_connect.py directly in ~/Databases using your editor. Run every python command below from this directory with the virtual environment active. If you open a new terminal, return to ~/Databases and activate .venv again.

Screenshot Required:

- docker_installation.png

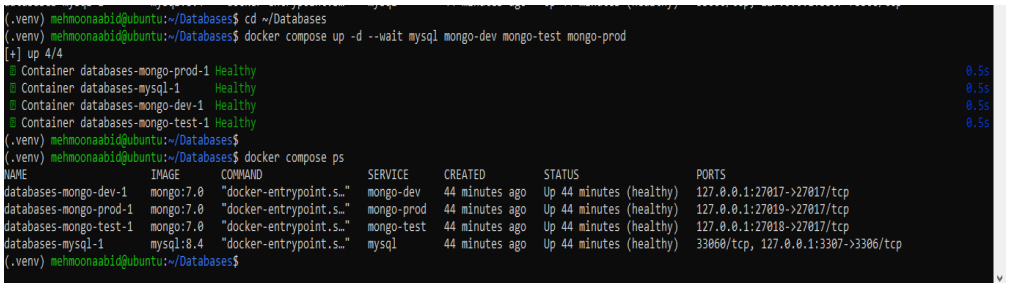


Task 3: Start the Database Services

1. Start the four supplied databases:
2. docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod

docker compose ps

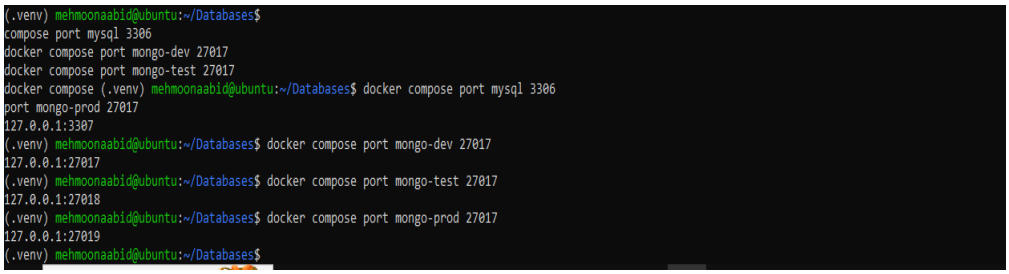
- Screenshot as database_services.png, showing all four services healthy.



3. Inspect the ports used by Python on the Ubuntu host:
4. docker compose port mysql 3306
5. docker compose port mongo-dev 27017
6. docker compose port mongo-test 27017

docker compose port mongo-prod 27017

- Screenshot as database_ports.png.



Service	Host port	Database	Purpose
MySQL	3307	school_mysql	MySQL connection exercise
MongoDB dev	27017	school_dev	Development environment
MongoDB test	27018	school_test	Testing environment
MongoDB prod	27019	school_prod	Production simulation

These public dummy values are supplied for the local lab. Keep this table outside screenshots. The initialization scripts create a small environment_info table/collection containing the actual environment name.

Task 4: MySQL with Values in the Code

1. Open db_connect.py in your editor:

nano db_connect.py

The starter contains this short MySQL connection:

```
import mysql.connector


connection = mysql.connector.connect(
    host="127.0.0.1",
```

```
    port=3307,
    user="mysql_student",
    password="MysqlDemoOnly123!",
    database="school_mysql",
)

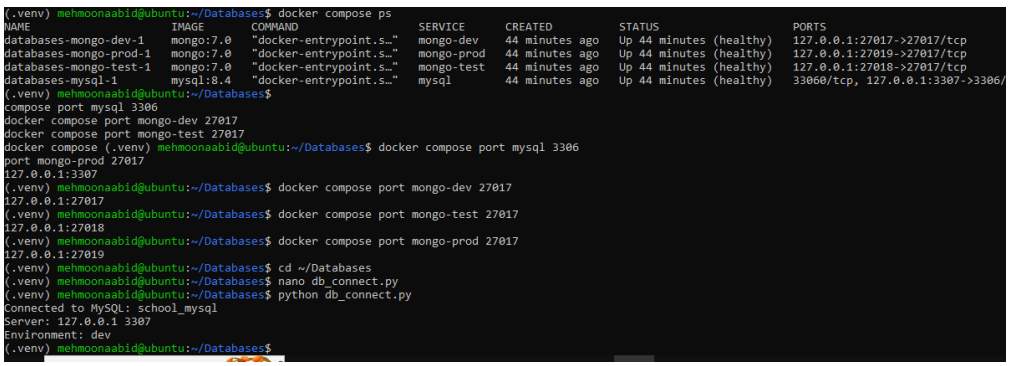

print("Connected to MySQL:", connection.database)
print("Server:", connection.server_host, connection.server_port)
cursor = connection.cursor()
cursor.execute("SELECT environment FROM environment_info")
print("Environment:", cursor.fetchone()[0])
cursor.close()
connection.close()
```

user and password supply the database credentials. host identifies the server, port identifies its MySQL service, and database selects the database. mysql.connector.connect() opens the connection.

2. Run the file:

```
python db_connect.py
```

- Screenshot as mysql_hardcoded_connection.png.



Show Connected to MySQL: school_mysql, server 127.0.0.1 3307, and Environment: dev.

The small SQL query reads the environment value stored in MySQL. cursor.close() and connection.close() release the resources. Record the result in your solution PDF.

Task 5: MongoDB with Values in the Code

1. Replace the contents of **the same db_connect.py** with:

```
from pymongo import MongoClient


username = "dev_user"

password = "DevDemoOnly123!"

endpoint = "127.0.0.1:27017"

database_name = "school_dev"


uri = "mongodb://" + username + ":" + password + "@" +
endpoint + "/" + database_name


client = MongoClient(uri, serverSelectionTimeoutMS=5000)

database = client[database_name]

database.command("ping")

print("Connected to MongoDB:", database.name)

print("Server:", client.address)

print("Environment:",
database.environment_info.find_one()["name"])

client.close()
```

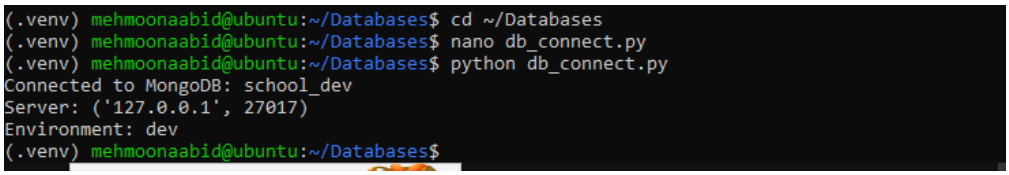
MongoClient(uri) creates the MongoDB client, and client[database_name] selects the database.

username, password, endpoint and database_name hold the settings separately. The + operator joins them into uri. The URI's database is also the authentication database in this lab.

2. Run the same command:

python db_connect.py

- Screenshot as mongo_hardcoded_connection.png.



Show database school_dev, port 27017, and environment dev.

database.command("ping") sends a real request; creating a client or selecting a database alone does not verify connectivity. The final query reads the environment from the selected database. serverSelectionTimeoutMS=5000 limits server selection to five seconds.

Task 6: MongoDB with os.getenv()

1. Replace the same file with the environment-variable version:

import os

from pymongo import MongoClient

username = os.getenv("MONGO_USERNAME")

password = os.getenv("MONGO_PASSWORD")

endpoint = os.getenv("MONGO_ENDPOINT")

database_name = os.getenv("MONGO_DATABASE")

if not username or not password or not endpoint or not database_name:

    raise SystemExit("Set all four MongoDB environment variables first.")

uri = "mongodb://" + username + ":" + password + "@" + endpoint + "/" + database_name

client = MongoClient(uri, serverSelectionTimeoutMS=5000)

database = client[database_name]

database.command("ping")

print("Connected to MongoDB:", database.name)

print("Server:", client.address)

print("Environment:", database.environment_info.find_one()["name"])

client.close()

Add import os and read the username, password, endpoint and database name separately using os.getenv(). Keep the URI concatenation, MongoDB connection and output block unchanged. The short if statement tells you if a required variable is missing.

2. Set the dev values in the same Ubuntu shell and run:
3. export MONGO_USERNAME=dev_user
4. export MONGO_PASSWORD='DevDemoOnly123!'
5. export MONGO_ENDPOINT=127.0.0.1:27017
6. export MONGO_DATABASE=school_dev
7. clear

python db_connect.py

- Screenshot as mongo_env_dev.png, showing school_dev, port 27017, and environment dev.

8. Set the test values and run:
9. export MONGO_USERNAME=test_user
10. export MONGO_PASSWORD='TestDemoOnly123!'
11. export MONGO_ENDPOINT=127.0.0.1:27018
12. export MONGO_DATABASE=school_test
13. clear

python db_connect.py

- Screenshot as mongo_env_test.png, showing school_test, port 27018, and environment test.

14. Set the simulated prod values and run:
15. export MONGO_USERNAME=prod_user
16. export MONGO_PASSWORD='ProdDemoOnly123!'
17. export MONGO_ENDPOINT=127.0.0.1:27019
18. export MONGO_DATABASE=school_prod
19. clear

python db_connect.py

- Screenshot as mongo_env_prod.png, showing school_prod, port 27019, and environment prod.

These export commands contain the supplied public dummy credentials. clear removes them from the visible terminal before you capture the result. Do not print MONGO_PASSWORD, the assembled URI or capture the exports in a screenshot.

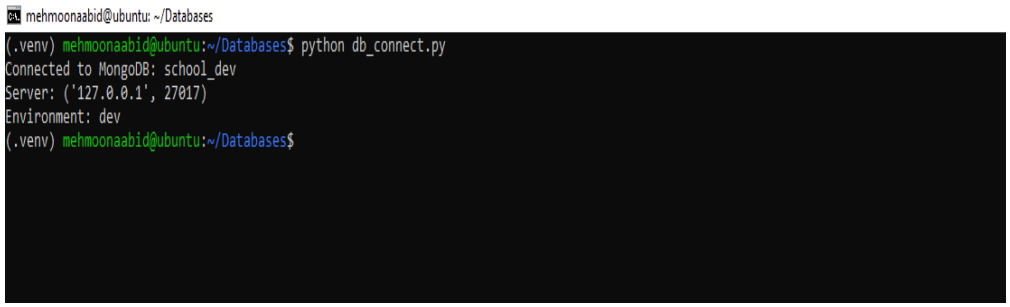
Python inherits exported variables from the shell. MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT and MONGO_DATABASE supply separate values. Python joins them into the connection URI.

Export all four matching values for each environment. This version does not read APP_ENV.

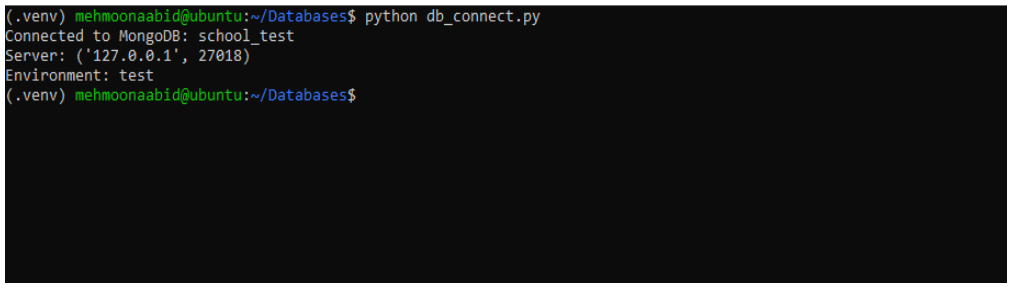
Complete **Exam Question 1** while this version is still in your file.

Screenshot Required:

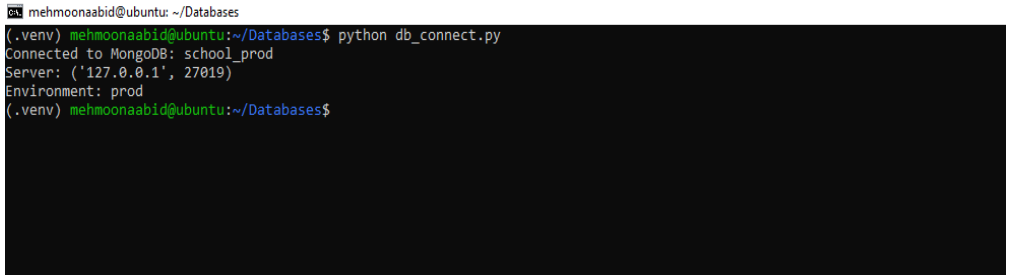
- mongo_env_dev.png



- mongo_env_test.png



- mongo_env_prod.png



.

Task 7: MongoDB with a Credentials File

1. Create one local file containing the three environment sections:
2. cp -n credentials.ini.example credentials.ini

ls credentials.ini

- Screenshot as credential_files_created.png.

The file contains:

Public dummy values for the disposable classroom databases only.

[dev]

username = dev_user

password = DevDemoOnly123!

endpoint = 127.0.0.1:27017

database = school_dev

[test]

username = test_user

password = TestDemoOnly123!

endpoint = 127.0.0.1:27018

database = school_test

[prod]

username = prod_user

password = ProdDemoOnly123!

endpoint = 127.0.0.1:27019

database = school_prod

2. Restrict the file to your account and check its mode:
3. chmod 600 credentials.ini

stat -c '%a %n' credentials.ini

- Screenshot as credential_file_permissions.png, showing 600 credentials.ini.

4. Replace the contents of the same db_connect.py with:

```
import os
import configparser
from pymongo import MongoClient

config = configparser.ConfigParser(interpolation=None)
config.read("credentials.ini")
environment = os.getenv("APP_ENV", "dev")
username = config[environment]["username"]
password = config[environment]["password"]
endpoint = config[environment]["endpoint"]
database_name = config[environment]["database"]

uri = "mongodb://" + username + ":" + password + "@" +
endpoint + "/" + database_name

client = MongoClient(uri, serverSelectionTimeoutMS=5000)
database = client[database_name]
database.command("ping")
print("Connected to MongoDB:", database.name)
print("Server:", client.address)
```

print("Environment:",
database.environment_info.find_one()["name"])

client.close()

The configparser module is included with Python. config.read() reads the file, and config[environment] chooses a section. os.getenv("APP_ENV", "dev") uses the selected section name, with dev as its default.

The username, password, endpoint and database_name assignments read separate values from that section. Python builds uri with the same concatenation line used before. Keep the MongoDB connection and output block unchanged.

4. Choose dev and run:
5. export APP_ENV=dev

python db_connect.py

   - Screenshot as mongo_file_dev.png, showing the dev endpoint, database and environment.

6. Choose test and run:
7. export APP_ENV=test

python db_connect.py

   - Screenshot as mongo_file_test.png, showing the test endpoint, database and environment.

8. Choose simulated prod and run:
9. export APP_ENV=prod

python db_connect.py

   - Screenshot as mongo_file_prod.png, showing the prod endpoint, database and environment.

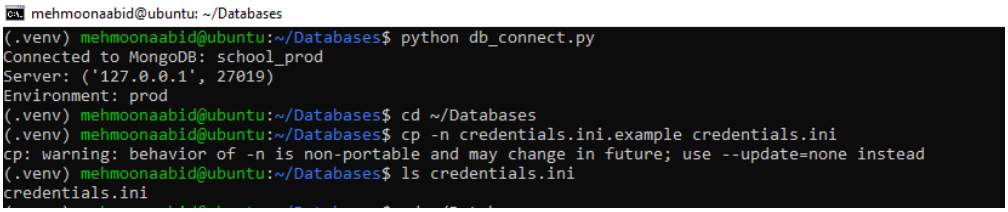
The file version reads APP_ENV to choose a section and loads the four connection settings from that section. An INI file stores plaintext. interpolation=None makes the file reader treat values

literally. The supplied dummy credentials are joined directly into the URI.

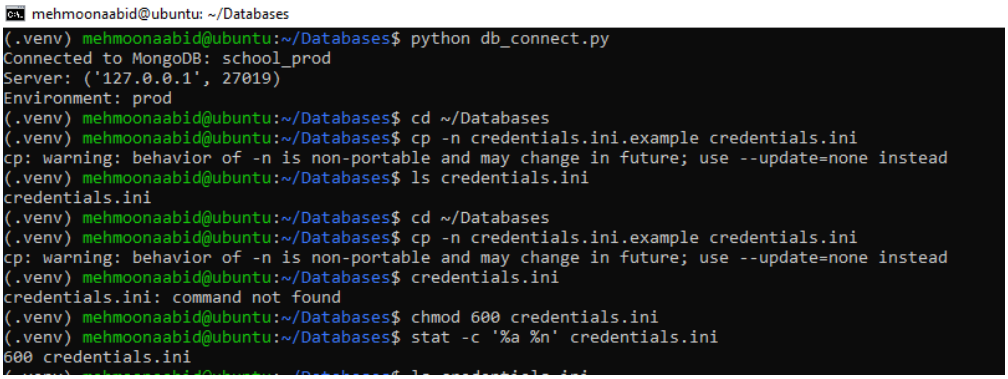
Complete **Exam Questions 2 and 3** after this task. Keep the file version as your final db_connect.py.

Screenshot Required:

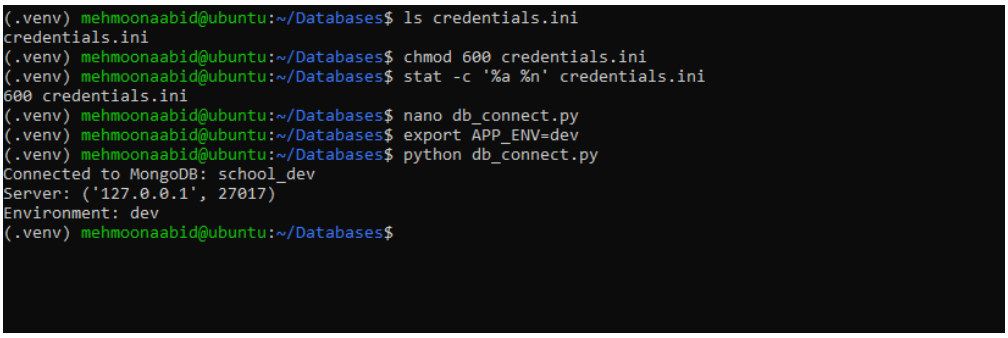
- credential_files_created.png



- credential_file_permissions.png



- mongo_file_dev.png



- mongo_file_test.png

```
(.venv) mehmoonaabid@ubuntu:~/Databases$ nano db_connect.py
(.venv) mehmoonaabid@ubuntu:~/Databases$ export APP_ENV=dev
(.venv) mehmoonaabid@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) mehmoonaabid@ubuntu:~/Databases$ export APP_ENV=test
(.venv) mehmoonaabid@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) mehmoonaabid@ubuntu:~/Databases$
```

- mongo_file_prod.png

```
(.venv) mehmoonaabid@ubuntu:~/Databases$ nano db_connect.py
(.venv) mehmoonaabid@ubuntu:~/Databases$ export APP_ENV=dev
(.venv) mehmoonaabid@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) mehmoonaabid@ubuntu:~/Databases$ export APP_ENV=test
(.venv) mehmoonaabid@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) mehmoonaabid@ubuntu:~/Databases$ export APP_ENV=prod
(.venv) mehmoonaabid@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) mehmoonaabid@ubuntu:~/Databases$
```

Exam Evaluation Questions

Write your predictions, results and explanations in Lab4_Solution.pdf. The three exam-practice screenshots are not included in the 15 mandatory checks.

1. A Missing Environment Variable

Complete this during Task 6, before replacing the environment-variable code.

1. With the valid prod profile exported, remove only the database-name variable:
2. unset MONGO_DATABASE

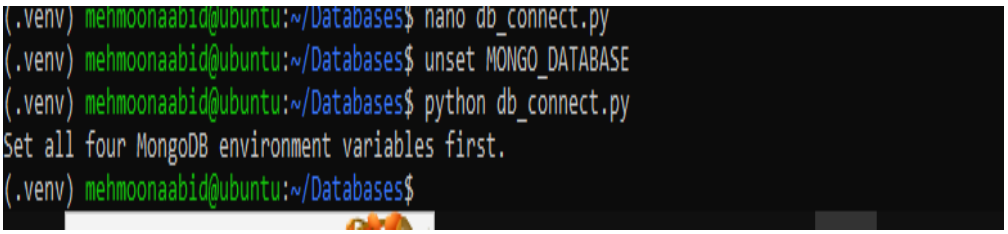
python db_connect.py

- Screenshot as Q1_missing_environment.png, showing Set all four MongoDB environment variables first..

3. Explain which os.getenv() call returned no value and why the if statement stopped the program.
4. Restore export MONGO_DATABASE=school_prod and confirm that the program connects again.

   Solution:

   The os.getenv("MONGO_DATABASE") function returns None if the environment variable is not set. The if statement checks whether any required MongoDB environment variable is missing. If a variable is missing, the program stops before connecting to the database. When the missing variable is restored, the program can connect successfully.



2. Which Source Does the File Version Read?

Complete this after Task 7.

1. Leave Task 6's prod username, password, endpoint and database variables exported. Remove only the section selector:
2. unset APP_ENV

python db_connect.py

   - Screenshot as Q2_file_source_selection.png, showing the dev connection.

3. Explain why this version connects to dev even though the shell still contains the prod connection settings.
4. Select APP_ENV=test and verify the test connection.

Solution:

The Task 7 version reads database credentials from the credentials.ini file. The APP_ENV variable selects which section of the file to use. If APP_ENV is not set, the program uses dev as the default environment. Therefore, it connects to the development database even when production environment variables are exported in the shell.

```
(.venv) mehmoonaabid@ubuntu:~/Databases$ nano db_connect.py
(.venv) mehmoonaabid@ubuntu:~/Databases$ unset APP_ENV
(.venv) mehmoonaabid@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
```

3. A Wrong Password in the File

1. Open the local credentials.ini. In the [test] section only, temporarily set password = wrong. Keep the username, endpoint and database name unchanged.
2. Run:
3. export APP_ENV=test

python db_connect.py

   - Screenshot as Q3_wrong_file_password.png, showing the authentication error without displaying the file.

4. Explain why selecting the right endpoint does not make an incorrect password valid.
5. Restore the supplied test dummy password, run again successfully, then restore APP_ENV=prod. Do not commit the credential file.

   Solution:
   MongoDB checks the username and password before allowing access to the database. Selecting the correct endpoint only identifies the database server; it does not make an incorrect password valid. Therefore, using the wrong password in the [test] section causes an

authentication error. After restoring the correct test password, the connection succeeds.

